

70. Chercher et changer une case

Ramasser une pièce, c'est trouver sa case dans le tableau, et la mettre à zéro.

1. Ce qu'il faut comprendre

Un tableau en mémoire de travail (sans `const`) **peut changer** pendant le jeu. Ici, `pieces[i]` vaut 1 s'il y a une pièce en colonne `i`, 0 si elle a été ramassée. L'écran n'est qu'un **reflet** du tableau : on modifie les données, puis on redessine.

À chaque appui sur A, on **cherche la première pièce** : une boucle, un `if`, un `break` — la recherche du chapitre 3, mais dans un tableau plutôt qu'à l'écran. On la met à 0.

Puis on **compte** ce qui reste : une boucle, un compteur. Remarque qu'on ne tient pas un compteur à part qu'il faudrait penser à diminuer : on **recompte** dans le tableau, qui est la seule vérité — et seulement **quand une pièce est ramassée**. Le comptage apparaît deux fois dans le programme (au premier dessin, puis après chaque ramassage) : c'est exactement le genre de répétition que les **fonctions**, au chapitre suivant, feront disparaître.

Le dessin complet se fait une fois, écran éteint. Ensuite, ramasser une pièce ne réécrit **qu'une case** : celle de la pièce. C'est la règle du VBlank — n'écrire que ce qui change.

`avant` sert à détecter **le moment où A s'enfonce** — le « front ». Sans lui, tenir A une demi-seconde ramasserait trente pièces d'un coup : une par image. On agit seulement quand A est enfoncé **maintenant** et ne l'était **pas** à l'image d'avant.

À toi : ramasse plutôt la **dernière** pièce (fais tourner la boucle de 9 vers 0).

Les fonctions de cette leçon — et le tuto de chacune

- `#include <ecran>` → [27.1. La fonction `ecran\(\)` — éteindre et rallumer l'écran](#)
- `#include <texte>` → [0.0.2. La fonction `texte\(\)` — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction `nombre\(\)` — écrire un nombre](#)
- `#include <bouton>` → [0.65.5. La fonction `bouton\(\)` — lire la manette](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **944 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	66 o	← la console, à l'allumage — la mise en route de la console
main()	282 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← <code>image()</code> — attend l'image suivante
AttendreVBlank	16 o	← <code>ecran()</code> — attend que l'écran soit libre
AttendreAcces	12 o	← <code>EcrireTexte</code> , <code>EcrireNombre</code> — attend de pouvoir écrire une case
AdresseCase	10 o	← <code>texte()</code> — trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← <code>texte()</code> — écrit un texte, lettre par lettre
EcrireNombre	33 o	← <code>nombre()</code> — écrit un nombre en chiffres

```

CopierTuiles      14 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte      14 o ← démarrage – vide les deux cartes de l'écran
LireManette       37 o ← bouton() – lit les huit boutons
Tuiles            304 o ← CopierTuiles – 19 lettres sur 43
"A: RAMASSER"     11 o ← texte() – un texte gravé
"RESTE:"          6 o ← texte() – un texte gravé
"0"               1 o ← texte() – un texte gravé
"."               1 o ← texte() – un texte gravé
"."               1 o ← texte() – un texte gravé
pieces_depart_0   10 o ← démarrage – une table gravée

Lettres gravées : AEMORST0123456789.:

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche,
DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :
    texte() × 5    85 o sur place, dans main() + AdresseCase (10 o), EcrireTexte (10 o)
    ecran() × 2    21 o sur place, dans main() + AttendreVBlank (16 o)
    nombre() × 2   20 o sur place, dans main() + EcrireNombre (33 o)
    bouton() × 1   12 o sur place, dans main() + LireManette (37 o)
    image() × 1    3 o sur place, dans main() + AttendreImage (28 o)

```

```
// CE PROGRAMME : une rangée de pièces 0. Chaque appui sur A ramasse la
// première pièce qui reste (elle devient un point), et le compteur baisse.
//
// Ce qui est nouveau : CHERCHER une case dans un tableau, puis la CHANGER.
// Le tableau est la vérité du jeu ; l'écran ne fait que la montrer.

#include <ecran>      // éteint ou rallume l'écran
#include <texte>       // écrit un texte à l'écran
#include <nombre>      // écrit un nombre en chiffres
#include <bouton>      // lit un bouton de la manette
```

[illegible]

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    // Le dessin complet, une fois, écran éteint.
    ecran(0);
    texte(1, 2, "A: RAMASSER");             // La consigne.
    texte(1, 11, "RESTE:");
    uint8_t reste = 0;                      // Combien de pièces restent.
    for (uint8_t i = 0; i < 10; i++) {      // Pour chaque case i (0 à 9) :
        if (pieces[i] == 1) {              // une pièce ?
            texte(1 + i * 2, 7, "0");       // on dessine un 0 (colonnes 1, 3, 5 ... 19),
            reste++;                        // et on la compte.
        } else {                           // pas de pièce :
            texte(1 + i * 2, 7, ".");       // un point.
        }
    }
    nombre(8, 11, reste);                  // Affiche 007.
    ecran(1);                             // On rallume.

    uint8_t avant = 0;                     // A était-elle enfoncée à l'image d'avant ?

    while (true) {                         // La boucle du jeu :
        image();                           // une image.

        uint8_t maintenant = bouton(A);    // A enfoncée maintenant ? (1 ou 0)
        if (maintenant && !avant) {        // Enfoncée maintenant ET pas avant :
                                            // c'est le tout début de l'appui.
                                            // (Sinon, une pièce partirait à CHAQUE image
                                            // tant que le doigt reste sur A.)

            // Chercher la première pièce.
            for (uint8_t i = 0; i < 10; i++) { // On regarde les cases dans l'ordre :
                if (pieces[i] == 1) {         // la première qui contient une pièce...
                    pieces[i] = 0;           // ramassée : la case passe à 0 dans le tableau,
                    texte(1 + i * 2, 7, "."); // une seule case change à l'écran,
                    break;                   // et on s'arrête : une seule pièce par appui.
                }
            }

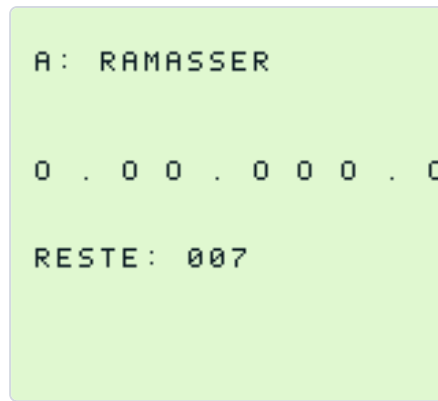
            // Recompter dans le tableau.
            reste = 0;                       // On repart de 0...
            for (uint8_t i = 0; i < 10; i++) {
                if (pieces[i] == 1) reste++; // ... et on ajoute 1 par pièce restante.
            }
            nombre(8, 11, reste);            // 006, puis 005, ... jusqu'à 000.
        }
        avant = maintenant;                 // On retient A pour l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran a changé

5. Ce qu'on doit voir

Une rangée de O et de points ; chaque appui sur A change le premier O en point, et le compte descend.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ, 7 pièces
- ✓ un appui ramasse une pièce
- ✓ c'est bien la première qui a disparu
- ✓ tenir A longtemps n'en ramasse qu'une de plus (005)

Ensuite : **71. Une grille : un tableau à deux dimensions** — chapitre 10 : Les tableaux : ranger beaucoup de valeurs. · gameboy3
— écrire une cartouche Game Boy en C++